



National Teachers College
629 J Nepomuceno, Quiapo, Manila, 1001 Metro Manila
Bachelor of Science in Information Technology

STUDENT TASK MANAGER SYSTEM

Submitted by :

Sadili, James Vincent P.
Aguelo, Andrei Francis B.
Balanoyos, Ryan Benedict

BSIT 1.3

Instructor:

Neypes, Justin Louise R.

Date:

May 17, 2026

Final Project/Exam: C++ Solution for Sustainable Development

Section 1: Introduction

1.1 The Student Task Manager System is a software application designed to help students organize their academic responsibilities such as assignments, quizzes, projects, and deadlines. The system allows users to add tasks, edit information, mark tasks as completed, and monitor upcoming deadlines in one organized platform.

This project aims to reduce academic stress and improve time management among students by providing a simple and efficient way to track school activities. The system promotes better learning habits and productivity by helping students manage their academic workload effectively. By improving organization and reducing missed deadlines, the software contributes to a more efficient and supportive learning environment.

1.2 Why Does This Software Need to Exist?

Many students struggle with managing their school tasks because of poor organization and busy schedules. Assignments and projects are sometimes forgotten, leading to late submissions, lower grades, and increased stress.

Traditional methods such as writing tasks on paper notes or relying only on memory are not always effective. Students often need a centralized system where they can easily store, update, and monitor their academic responsibilities.

The Student Task Manager System is needed to:

- Help students organize academic tasks efficiently
- Remind users about deadlines
- Reduce forgotten assignments and late submissions
- Improve productivity and time management
- Minimize academic stress through better planning

- **Section 2: Requirements Analysis**

2.1 Functional Requirements (FR): What the system *does* (e.g., "FR1: System must calculate carbon footprint").

These are the main functions that the **Student Manager System** can perform based on the program code.

FR1: View All Students

The system must allow users to display all student records stored in the database.

FR2: Add New Student

The system must allow users to add a new student by entering:

- Student name
- Course
- GPA
- Enrollment year
- Active status

FR3: Search Student

The system must allow users to search for students:

- By name
- By course

FR4: Update Student Information

The system must allow users to modify existing student information such as:

- Name
- Course
- GPA
- Active status

FR5: Delete Student

The system must allow users to delete a student record after confirmation.

FR6: Generate Student Report

The system must generate and display reports containing student information and statistics.

FR7: Save and Load Data

The system must save student records into the `students.txt` file and load existing data automatically when the program starts.

FR8: Filter Student Records

The system must allow users to filter and display:

- All students
- Active students only
- Scholarship-eligible students

2.2 Non-Functional Requirements (NFR): Performance, Robustness (Input Validation), and Readability.

Performance

The system should quickly load, search, update, and display student records without noticeable delays.

Robustness (Input Validation)

The system should validate all user inputs to prevent invalid or incorrect data. Examples include:

- Preventing empty names or course fields
- Accepting only valid GPA values between 0.0 and 4.0
- Accepting only valid integers for menu choices and student IDs
- Preventing invalid active status values other than 1 or 0

Readability

The source code should be easy to read and maintain by using:

- Proper indentation
- Clear function names
- Comments
- Organized menu structures

Reliability

The system should reliably save student data into the external file to avoid data loss when the program closes.

Usability

The system should provide a simple console-based menu interface that users can easily understand and navigate.

Maintainability

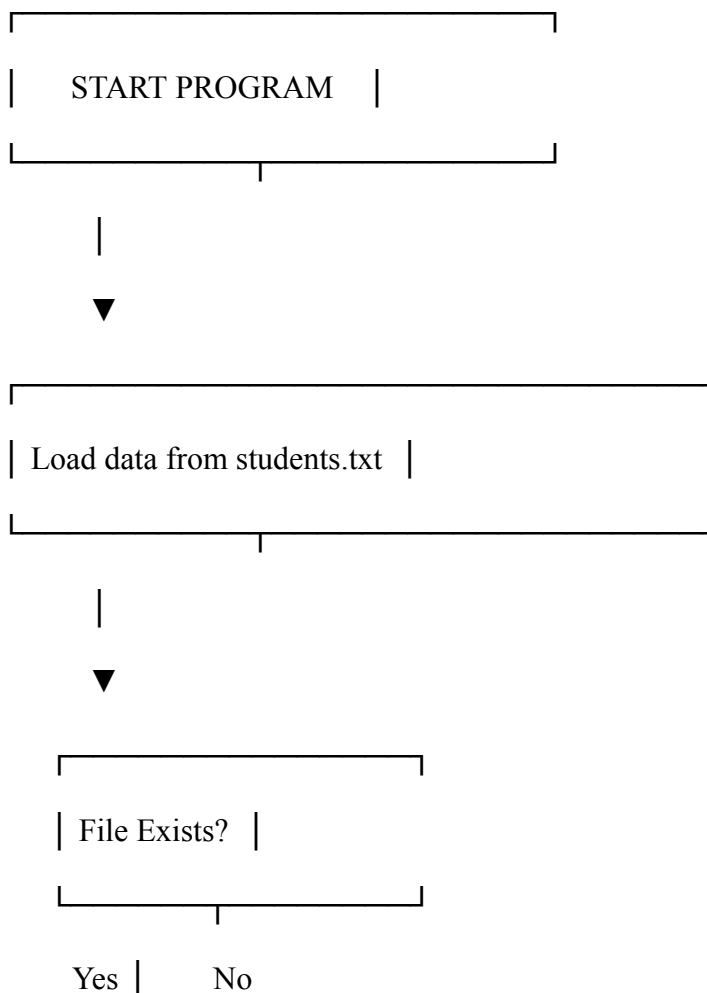
The program should use modular functions such as:

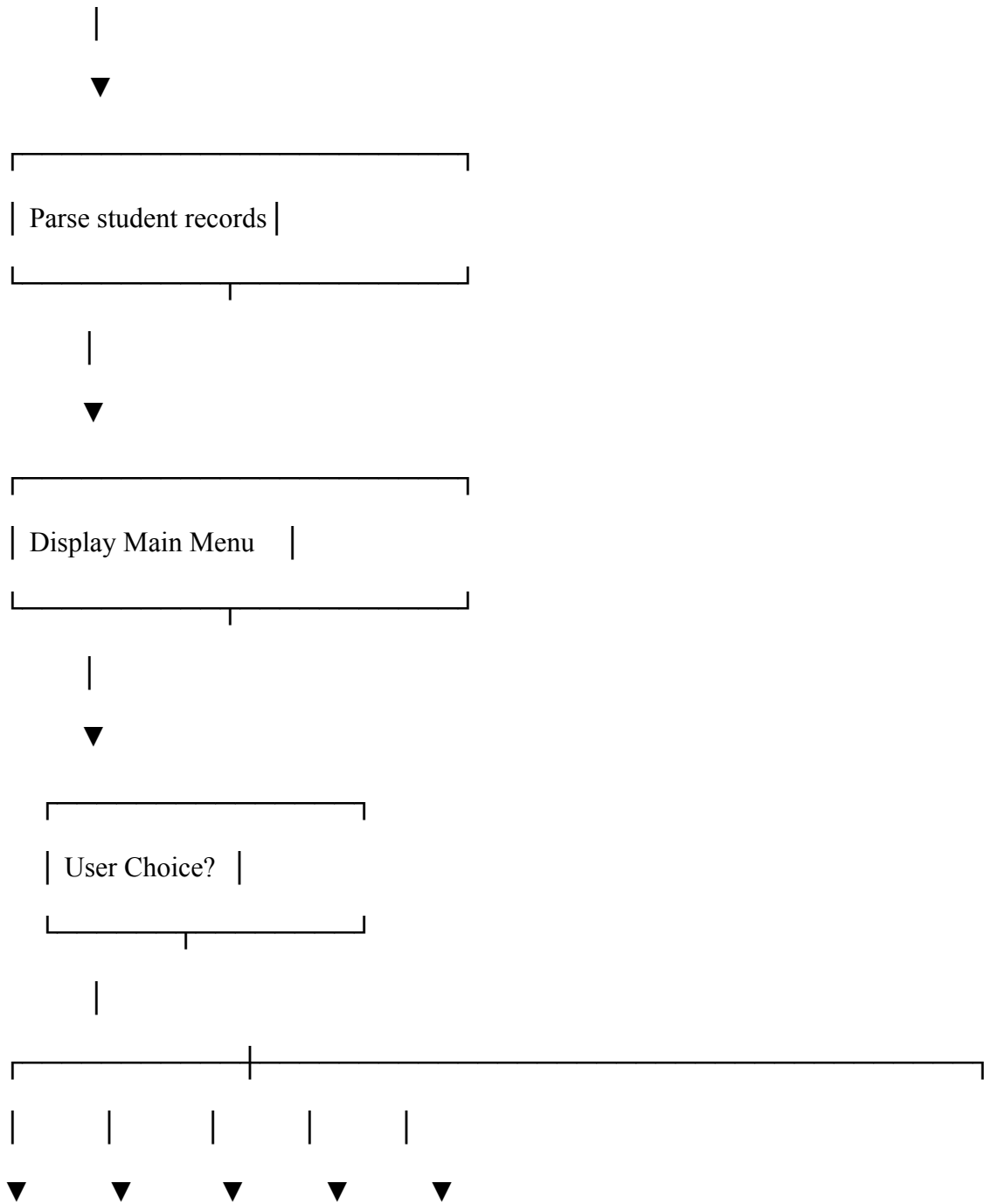
- `handleAddStudent()`
- `handleSearchStudent()`
- `handleUpdateStudent()`
- `validateInput()`

This makes the system easier to update and improve in the future.

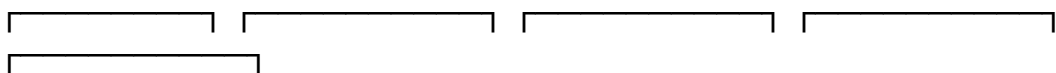
- **Section 3: Design Specification**

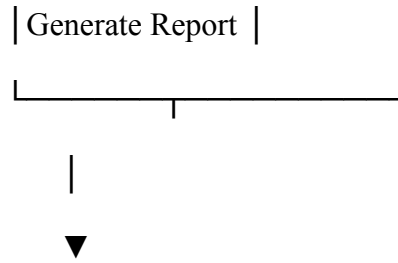
3.1 **Algorithmic Flowchart:** A detailed flowchart of the system's most complex logic.



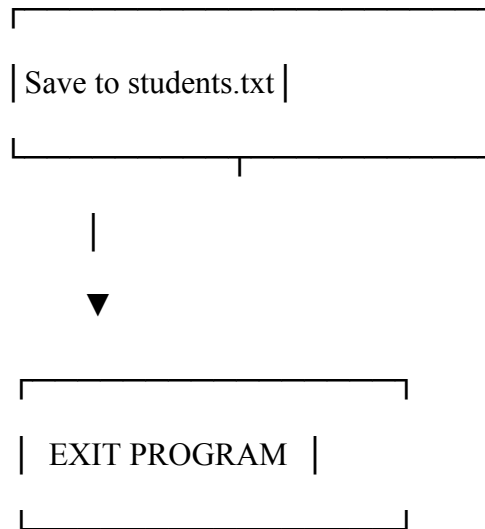


1. VIEW 2. ADD 3. SEARCH 4. UPDATE 5. DELETE

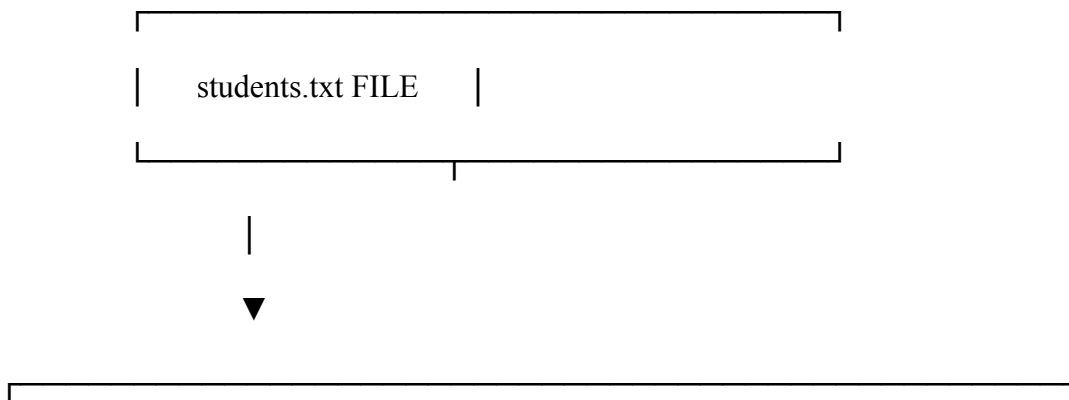




7. SAVE & EXIT



3.2 File Structure Diagram: Explanation of the data format used in external files.



STUDENT RECORD FORMAT						
StudentID	Name	Course	GPA	Active	Year	
Integer	String	String	Double	Boolean	Int	



EXAMPLE RECORD						
101	Juan Dela Cruz	BSIT	3.75	1	2022	
102	Maria Santos	BSCS	3.20	1	2021	
103	John Reyes	BSBA	2.85	0	2020	



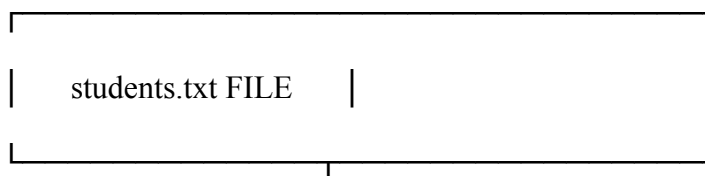
FIELD DESCRIPTION	
StudentID	Unique ID number of the student
Name	Full name of the student

Course	Student's course or program
GPA	Grade Point Average (0.0 - 4.0)
Active	1 = Active Student, 0 = Inactive Student
Year	Enrollment year



NOTES
• Each student record is stored in one line • Fields are separated using the pipe symbol " " • Data is loaded when the program starts • Updated data is saved when the program exits

- **3.2 File Structure Diagram:** Explanation of the data format used in external files.



STUDENT RECORD FORMAT

StudentID	Name	Course	GPA	Active	Year	
Integer	String	String	Double	Boolean	Int	

|



EXAMPLE RECORD						
101	Juan Dela Cruz	BSIT	3.75	1	2022	
102	Maria Santos	BSCS	3.20	1	2021	
103	John Reyes	BSBA	2.85	0	2020	

|



FIELD DESCRIPTION						
StudentID	Unique ID number of the student					
Name	Full name of the student					
Course	Student's course or program					

GPA	Grade Point Average (0.0 - 4.0)	
Active	1 = Active Student, 0 = Inactive Student	
Year	Enrollment year	

|



	NOTES	

	• Each student record is stored in one line	
	• Fields are separated using the pipe symbol " "	
	• Data is loaded when the program starts	
	• Updated data is saved when the program exits	

Student ID - A unique numeric ID for each students that automatically assigned by the system

Name - Full Name of a student

Course - Course or program that student is enrolled in

GPA - Grade point average ranging from 0.0 to 4.0

Active - Status of the student , 1=active, 0=inactive

Year- Year when the student was enrolled in